

# Microservices with Spring Boot 3 and Spring Cloud

## 1. Creating Microservices for Account and Loan

### Aim

To create two independent Spring Boot microservices (Account and Loan) that provide REST APIs for account and loan information.

### Tools Required

- Java 17
- Spring Boot 3
- Maven
- Spring Web
- Spring Boot DevTools
- Eclipse/IntelliJ IDEA
- Postman/Web Browser

### Procedure

#### Account Microservice

1. Create a Spring Boot project using Spring Initializr.
2. Add the dependencies:
  - Spring Web
  - Spring Boot DevTools
3. Set:
  - Group: `com.cognizant`
  - Artifact: `account`
4. Create a REST Controller.
5. Implement the endpoint:

GET /accounts/{number}

6. Return a dummy account object.

### Sample Response

```
{  
  
  "number": "00987987973432",  
  
  "type": "Savings",  
  
  "balance": 234343  
}
```

### Loan Microservice

1. Create another Spring Boot project.
2. Set:
  - Group: `com.cognizant`
  - Artifact: `loan`
3. Add Spring Web and DevTools.
4. Configure

`server.port=8081`

5. Implement the endpoint

`GET /loans/{number}`

### Sample Response

```
{  
  
  "number": "H00987987972342",  
  
  "type": "Car",  
  
  "loan": 400000,  
  
  "emi": 3258,  
  
  "tenure": 18  
}
```

## Output

Account Service

`http://localhost:8080/accounts/00987987973432`

Loan Service

`http://localhost:8081/loans/H00987987972342`

## Result

The Account and Loan Microservices were created successfully as two independent REST APIs running on different ports.

# 2. Create Eureka Discovery Server and Register Microservices

## Aim

To create a Eureka Discovery Server and register the Account and Loan microservices with it.

## Tools Required

- Spring Boot 3
- Spring Cloud Eureka Server
- Spring Cloud Eureka Client
- Maven

## Procedure

### Step 1: Create Eureka Discovery Server

1. Create a Spring Boot project.
2. Add dependency:
  - Eureka Server
3. Configure the application properties.

`server.port=8761`

`eureka.client.register-with-eureka=false`

eureka.client.fetch-registry=false

logging.level.com.netflix.eureka=OFF

logging.level.com.netflix.discovery=OFF

4. Add the annotation

@EnableEurekaServer

5. Run the application.
6. Open

<http://localhost:8761>

The Eureka dashboard should open.

## Step 2: Register Account Service

1. Add dependency
  - Eureka Discovery Client
2. Add annotation

@EnableDiscoveryClient

3. Configure

spring.application.name=account-service

4. Start Eureka Server.
5. Start Account Service.
6. Refresh

<http://localhost:8761>

The Account Service will appear in the registry.

## Step 3: Register Loan Service

Repeat the same steps for the Loan Service.

Configuration

spring.application.name=loan-service

Start the Loan Service.

Refresh Eureka Dashboard.

Both services should now be registered.

## **Output**

Eureka Dashboard

Registered Services

ACCOUNT-SERVICE

LOAN-SERVICE

## **Result**

The Eureka Discovery Server was created successfully, and both Account and Loan Microservices were registered and discovered through Eureka.